

INTERNSHIP TASK REPORT

Cyber Security Internship

Finding weak spots in a target machine and reading what travels across the network

Name	Muhammad Ali Mobeen
Student ID	B7/9
Position	Cyber Security Intern (Remote Internship in Cyber Security)
Organisation	Progree, Islamabad, Pakistan
Internship period	20 September 2026 to 20 October 2026
Tasks covered	Task 2: Host & Network Vulnerability Assessment Task 3: Packet Inspection & Protocol Analysis
Date of report	1 October 2026

All testing was done in an isolated practice lab on a deliberately vulnerable virtual machine. Please read the Disclaimer and Limitation of Liability on page 3.

Contents

Disclaimer and Limitation of Liability..... 3

1. Summary.....4

2. Lab Setup and Approach.....5

3. Task 2: Host & Network Vulnerability Assessment.....6

4. Task 3: Low-Level Network Packet Inspection & Protocol Analysis.....15

5. Conclusion and What I Learned.....23

Appendix A: Evidence Index.....24

Appendix B: Task Coverage.....26

A note on the figures

The report has 23 numbered figures: Figures 1 to 9 belong to Task 2 and Figures 10 to 23 belong to Task 3. Each one is a screenshot I took during the lab work, placed next to the finding it supports. Appendix A lists every screenshot with its file name.

Disclaimer and Limitation of Liability

Please read this page before using or sharing the report.

1. Purpose of this report

I wrote this report as part of the Progree Remote Internship in Cyber Security. It records the practical work I did for Task 2 and Task 3, and it is meant for learning and for evaluation by the Progree team.

2. Where and how the testing was done

Every scan and packet capture was carried out by me inside an isolated virtual lab, against Metasploitable 2, a virtual machine that is intentionally vulnerable and designed for practice. I did not scan, test or capture traffic from any production system, third-party network or internet service.

3. Educational use only

The tools and techniques described here must only be used on systems you own or have clear written permission to test. Accessing computer systems without authorisation is a criminal offence in most countries, including under Pakistan's Prevention of Electronic Crimes Act, 2016. I do not support or take responsibility for any use of this material outside a lawful, authorised setting.

4. Accuracy and limits of the findings

The results are a snapshot taken on 1 October 2026. The Nessus scan was unauthenticated, and automated scanners can report false positives and miss real problems. I have done my best to describe the results correctly, but I cannot promise that every statement is complete or error-free. Items marked "verify" were not confirmed by hand.

5. No guarantee and limitation of liability

This report is not a guarantee that any system is secure, or insecure. To the fullest extent permitted by law, neither I nor Progree accept responsibility for any loss, damage or legal consequence that results from using, relying on or misusing the information or recommendations in this report. Anyone who acts on them should first verify them independently and test changes in a safe environment.

6. Credentials and personal data

The usernames and passwords visible in the screenshots (for example msfadmin / msfadmin and the VNC password "password") are the publicly known default logins of the practice VM. They are not real accounts, and no personal data belonging to other people was collected.

7. Third-party software

Nessus, Nmap, Nikto, Wireshark, Kali Linux, VirtualBox and any other products named in this report belong to their respective owners. I used them for learning, within their licences (Nessus was used on a trial licence), and show their screens only as evidence of my work.

1. Summary

During my Progree internship I worked on two hands-on security tasks. In **Task 2** I looked for weaknesses in a target machine using Nmap, Nessus and Nikto. In **Task 3** I captured the network traffic with Wireshark and studied what an observer on the network could see. This report explains what I did, what I found, and what I would recommend.

Everything was done in an isolated VirtualBox lab. My own machine was Kali Linux 2026.2 (10.0.0.4), and the target was Metasploitable 2 (10.0.0.16), a virtual machine that is built to be vulnerable so that people can practise on it safely.

1.1 What I found, in short

Area	Result
Open ports	Nmap found 30 open TCP ports. I documented 25 services in Section 3.4, and many of them run very old software such as vsftpd 2.3.4, OpenSSH 4.7p1, Apache 2.2.8, PHP 5.2.4 and Tomcat 5.5.
Nessus scan	193 findings: 10 Critical, 3 High, 32 Medium, 8 Low and 140 Informational . The scan took 22 minutes and was unauthenticated.
Nikto web scan	31 findings on the web server (port 80), including an exposed phpinfo() page, reachable phpMyAdmin files, browsable directories, an active TRACE method and missing security headers.
Wireshark analysis	FTP and HTTP traffic was completely readable. I captured the FTP username and password (msfadmin / msfadmin) in clear text and followed a simulated 10 MB data exfiltration from start to finish.

1.2 The findings that worried me most

- **A root shell that needs no password** is listening on port 1524. Nessus confirmed it by running as uid=0 (root).
- **The UnrealIRCd service has a known backdoor** (CVE-2010-2075), and the **VNC service on port 5900 uses the password "password"**.
- **Predictable encryption keys** caused by the Debian OpenSSL flaw (CVE-2008-0166) affect SSH, SMTP and PostgreSQL.
- **Nothing here is supported any more**. Ubuntu 8.04, Tomcat 5.5 (end of life since 30 September 2012), Apache 2.2.8 and PHP 5.2.4 no longer receive security fixes.
- **FTP, HTTP, Telnet, rlogin and rsh send everything in clear text**, which Task 3 showed directly in the packets.

In plain words: a stranger on the network could take over this machine in several different ways without knowing any password. That is expected for a practice machine, and it is exactly why it is useful for learning. A real server in this state would be taken offline and rebuilt, not patched.

2. Lab Setup and Approach

2.1 My lab

Item	Details
My machine	Kali Linux 2026.2 in VirtualBox. IP 10.0.0.4, MAC 08:00:27:3D:4D:96
Target	Metasploitable 2 (NetBIOS name METASPLOITABLE). IP 10.0.0.16, MAC 08:00:27:59:5B:04. Linux kernel 2.6 on Ubuntu 8.04 (Hardy)
Network	Isolated lab network 10.0.0.0/24. Host discovery found six live hosts, but I scanned only 10.0.0.16
Nmap	Version 7.99, run as: nmap -sV -p- 10.0.0.16
Nessus	Tenable Nessus Expert 10.12.4 (trial), "Basic Network Scan" policy, local scanner
Nikto	Version 2.6.1, run as: nikto -h http://10.0.0.16 -o nikto_scan.html -Format html
Wireshark	Version 4.6.6, capturing on eth0, saved as wireshark_eth0JWT5V3.pcapng

2.2 Ground rules I followed

I kept every action inside the lab and aimed only at the single target, 10.0.0.16. In Task 2 I did not run any exploit; I only discovered, listed and identified vulnerabilities. In Task 3 the only active steps were logging in to FTP with the practice VM's well-known default account and uploading a 10 MB test file, both to create traffic I could study.

2.3 How I worked

- Prepare:** install and activate Nessus and open Wireshark on Kali.
- Discover:** find the live hosts, then map every open port and service version.
- Assess:** run the Nessus and Nikto scans and collect the CVE references.
- Rank:** rate each problem by severity, how easy it is to abuse and how exposed it is, then decide what to fix first.
- Capture:** record the traffic, study the TCP handshake, spot the clear-text protocols and pick out the simulated exfiltration.
- Write up:** keep a screenshot for every important step and explain the result in plain language.

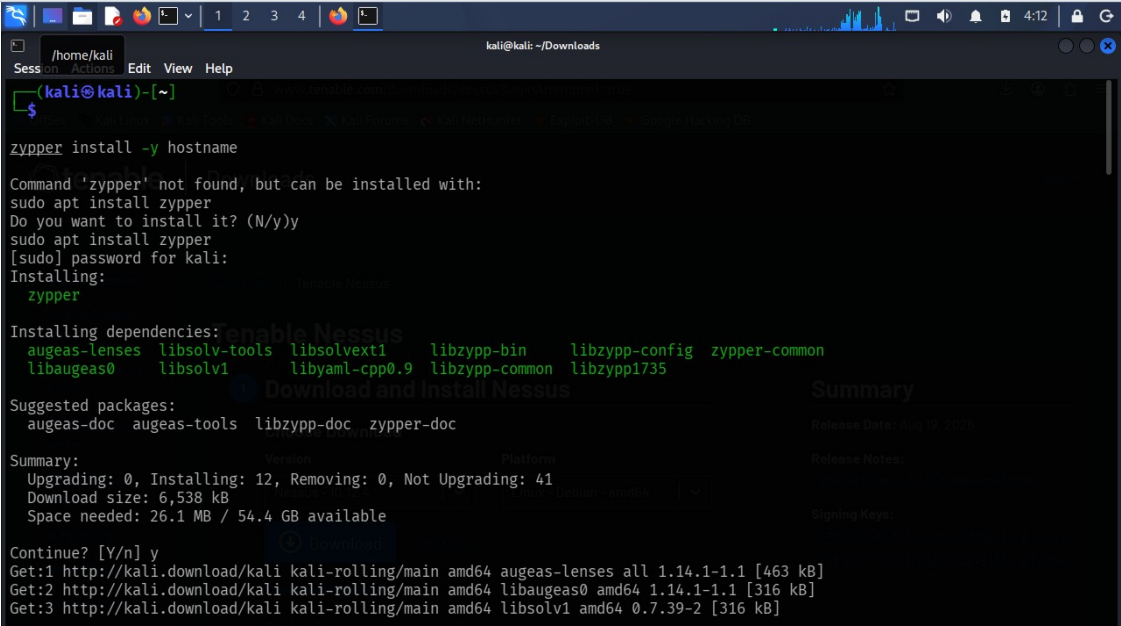
3. Task 2: Host & Network Vulnerability Assessment

3.1 What the task asked for

The task asked me to run reconnaissance tools in an isolated environment, map the open ports, check the software versions against known CVEs, and prepare a triage report. I used all three suggested tools (Nmap, Nessus and Nikto) so that I could compare their results instead of trusting a single scanner.

3.2 Getting Nessus ready

I began by installing Nessus on Kali. The package needed zypper first (Figure 1), and after that I installed Nessus 10.12.4 with dpkg. All of the cryptographic self-tests passed (Figure 2). Nessus asks for an activation code by e-mail, so I used a disposable address for the trial (Figure 3). Once activated, Nessus spent some time compiling its plugins and warned me that features would be limited until that finished (Figure 4).



```

kali@kali: ~/Downloads
(kali@kali)~$ zypper install -y hostname
Command 'zypper' not found, but can be installed with:
sudo apt install zypper
Do you want to install it? (N/y)y
sudo apt install zypper
[sudo] password for kali:
Installing:
zypper

Installing dependencies:
augeas-lenses libsolvable libsolvext1 libzypp-bin libzypp-config zypper-common
libaugeas0 libsolvl libyaml-cpp0.9 libzypp-common libzypp1735

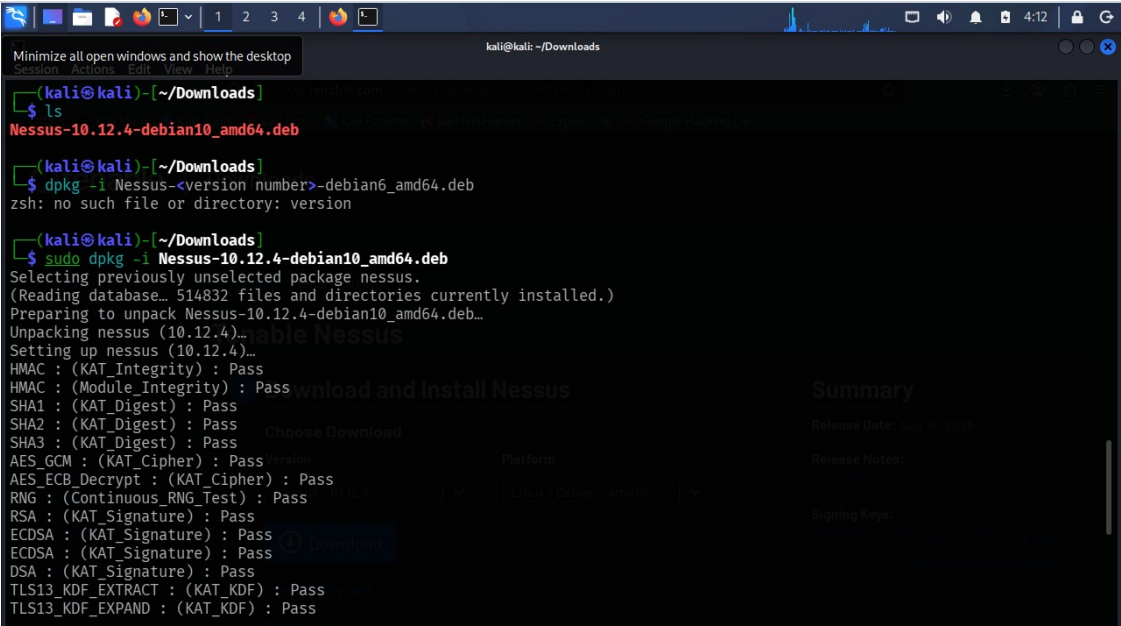
Suggested packages:
augeas-doc augeas-tools libzypp-doc zypper-doc

Summary:
Upgrading: 0, Installing: 12, Removing: 0, Not Upgrading: 41
Download size: 6,538 kB
Space needed: 26.1 MB / 54.4 GB available

Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 augeas-lenses all 1.14.1-1.1 [463 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libaugeas0 amd64 1.14.1-1.1 [316 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libsolvl amd64 0.7.39-2 [316 kB]

```

Figure 1. Installing the zypper prerequisite on Kali Linux before installing Nessus.



```

kali@kali: ~/Downloads
(kali@kali)~$ ls
Nessus-10.12.4-debian10_amd64.deb

(kali@kali)~$ dpkg -i Nessus-10.12.4-debian10_amd64.deb
zsh: no such file or directory: version

(kali@kali)~$ sudo dpkg -i Nessus-10.12.4-debian10_amd64.deb
Selecting previously unselected package nessus.
(Reading database... 514832 files and directories currently installed.)
Preparing to unpack Nessus-10.12.4-debian10_amd64.deb...
Unpacking nessus (10.12.4)...
Setting up nessus (10.12.4)...
HMAC : (KAT_Integrity) : Pass
HMAC : (Module_Integrity) : Pass
SHA1 : (KAT_Digest) : Pass
SHA2 : (KAT_Digest) : Pass
SHA3 : (KAT_Digest) : Pass
AES_GCM : (KAT_Cipher) : Pass
AES_ECB_Decrypt : (KAT_Cipher) : Pass
RNG : (Continuous_RNG_Test) : Pass
RSA : (KAT_Signature) : Pass
ECDSA : (KAT_Signature) : Pass
ECDSA : (KAT_Signature) : Pass
DSA : (KAT_Signature) : Pass
TLS13_KDF_EXTRACT : (KAT_KDF) : Pass
TLS13_KDF_EXPAND : (KAT_KDF) : Pass

```

Figure 2. Installing Nessus 10.12.4 with dpkg; all cryptographic self-tests report Pass.

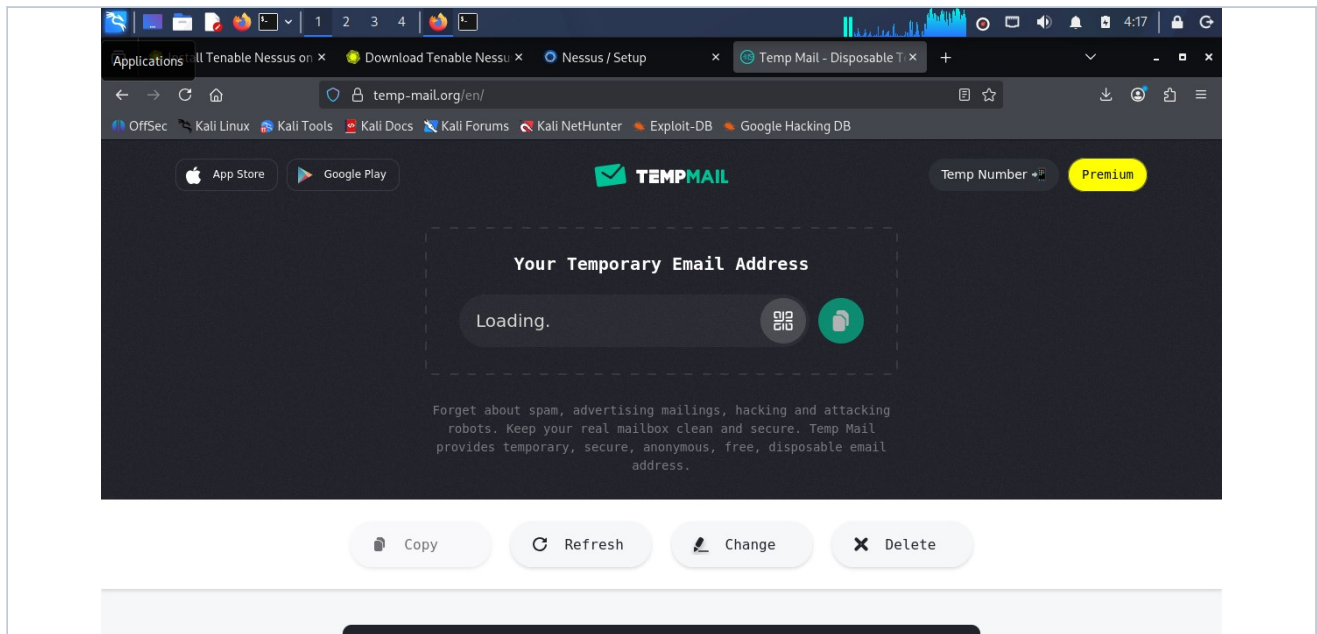


Figure 3. Disposable e-mail address used to receive the Nessus trial activation code.

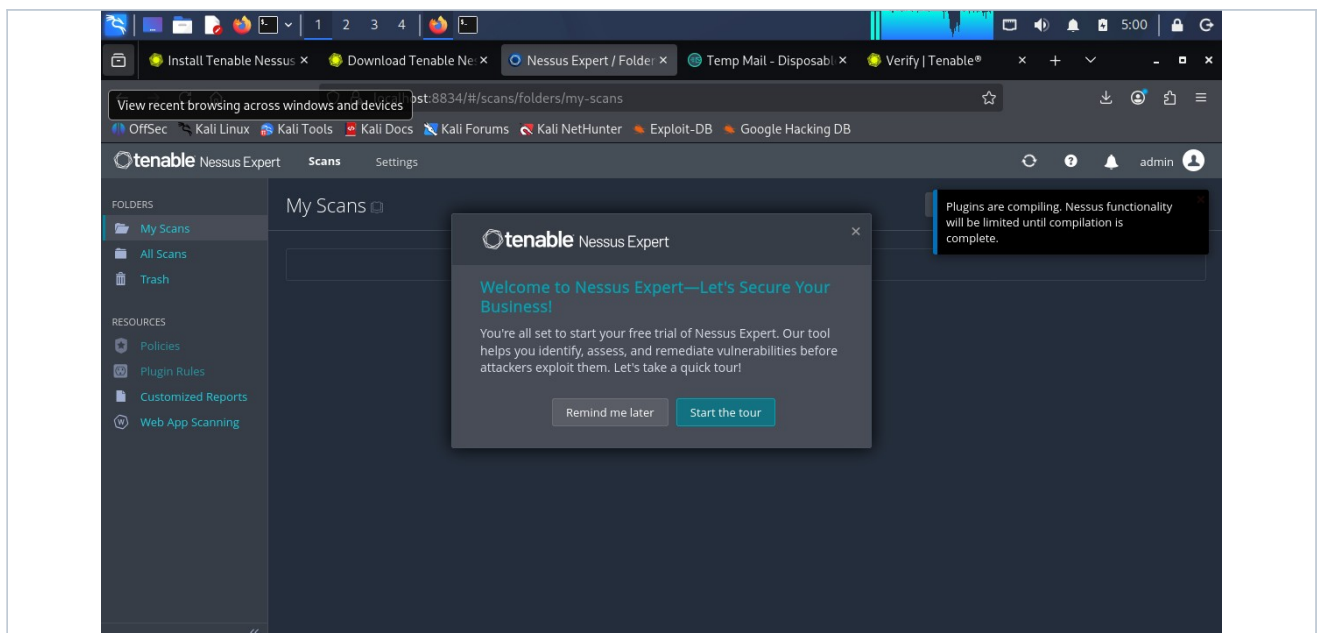


Figure 4. Nessus Expert first-run screen; plugins are compiling (functionality limited until complete).

3.3 Finding and mapping the target

I ran Nessus host discovery against 10.0.0.0/24, and it returned six live addresses: 10.0.0.1, .2, .4, .7, .10 and .16 (Figure 5). I picked only 10.0.0.16, the Metasploitable 2 machine. Its MAC address (08:00:27:59:5B:04) shows it is a VirtualBox guest.

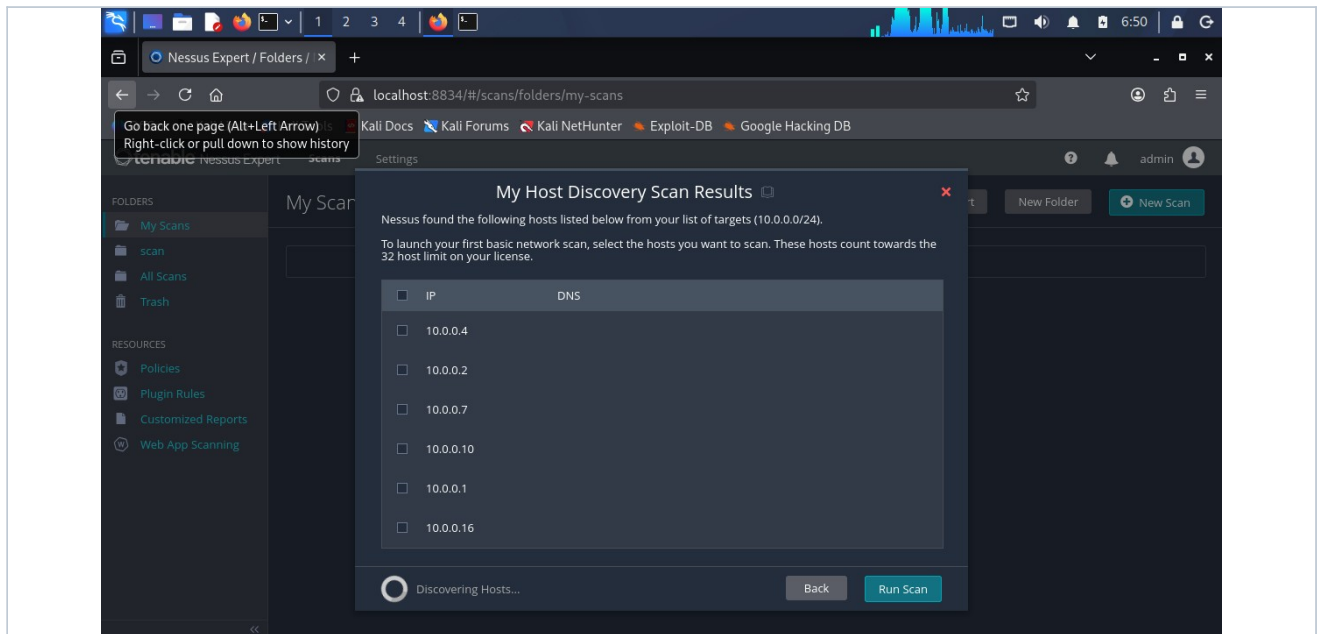


Figure 5. Nessus host discovery of 10.0.0.0/24 listing live hosts, including the target 10.0.0.16.

Next I scanned every TCP port with service detection (`nmap -sV -p-`). Nmap reported the host as up and said that **65,505 ports were closed, which leaves 30 open ports**. Figure 6 shows the first 22 services; my screenshot ends at port 6667.

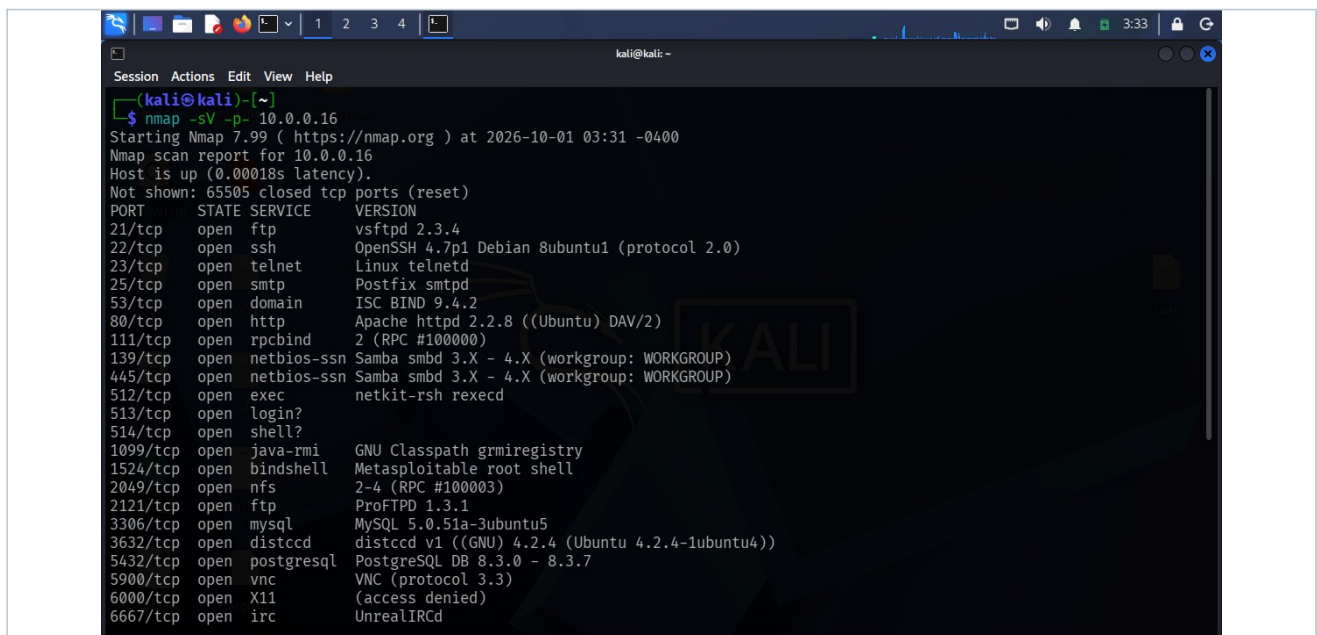


Figure 6. Nmap service-version scan (`nmap -sV -p- 10.0.0.16`) listing open ports and software versions.

3.4 Open ports and services

Table 1 combines the Nmap output with the Nessus port list. Because my screenshot stops at 6667, I took three higher ports (8009, 8180 and 8787) from the Nessus report. Nmap counted 30 open ports in total, so a few higher ports are not listed here because I did not capture them.

Table 1. Open ports and detected services on 10.0.0.16

Port	Service	Detected software / version	Source
21/tcp	FTP	vsftpd 2.3.4	Nmap
22/tcp	SSH	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)	Nmap

Port	Service	Detected software / version	Source
23/tcp	Telnet	Linux telnetd (unencrypted)	Nmap
25/tcp	SMTP	Postfix smtpd	Nmap
53/tcp, udp	DNS	ISC BIND 9.4.2	Nmap
80/tcp	HTTP	Apache httpd 2.2.8 (Ubuntu) DAV/2, PHP 5.2.4	Nmap, Nikto
111/tcp	RPC portmapper	rpcbind 2 (RPC #100000)	Nmap
139, 445/tcp	NetBIOS / SMB	Samba smbd 3.X to 4.X (workgroup WORKGROUP)	Nmap
512/tcp	rexec	netkit-rsh rexecd	Nmap
513/tcp	rlogin	rlogin service	Nmap, Nessus
514/tcp	rsh	rsh service	Nmap, Nessus
1099/tcp	Java RMI	GNU Classpath grmiregistry	Nmap
1524/tcp	Bind shell	Metasploitable root shell	Nmap, Nessus
2049/tcp	NFS	NFS v2 to v4 (RPC #100003)	Nmap
2121/tcp	FTP	ProFTPD 1.3.1	Nmap
3306/tcp	MySQL	MySQL 5.0.51a-3ubuntu5	Nmap
3632/tcp	distccd	distcc v1 ((GNU) 4.2.4, Ubuntu 4.2.4-1ubuntu4)	Nmap
5432/tcp	PostgreSQL	PostgreSQL DB 8.3.0 to 8.3.7	Nmap
5900/tcp	VNC	VNC (protocol 3.3)	Nmap, Nessus
6000/tcp	X11	X server (access denied)	Nmap, Nessus
6667/tcp	IRC	UnrealIRCd	Nmap, Nessus
8009/tcp	AJP13	Apache Tomcat AJP connector	Nessus
8180/tcp	HTTP	Apache Tomcat 5.5	Nessus
8787/tcp	Ruby DRb	Ruby Distributed Ruby (DRb) service	Nessus

3.5 What Nessus reported

The Nessus Basic Network Scan ran from 3:42 AM to 4:04 AM, about 22 minutes. The Auth column says **Fail**, which tells me the scan was unauthenticated: Nessus looked at the machine from the network only and could not run local patch checks. I took Figure 7 while the scan was still running, so those numbers were provisional. The final result is in Figure 8 and Table 2.

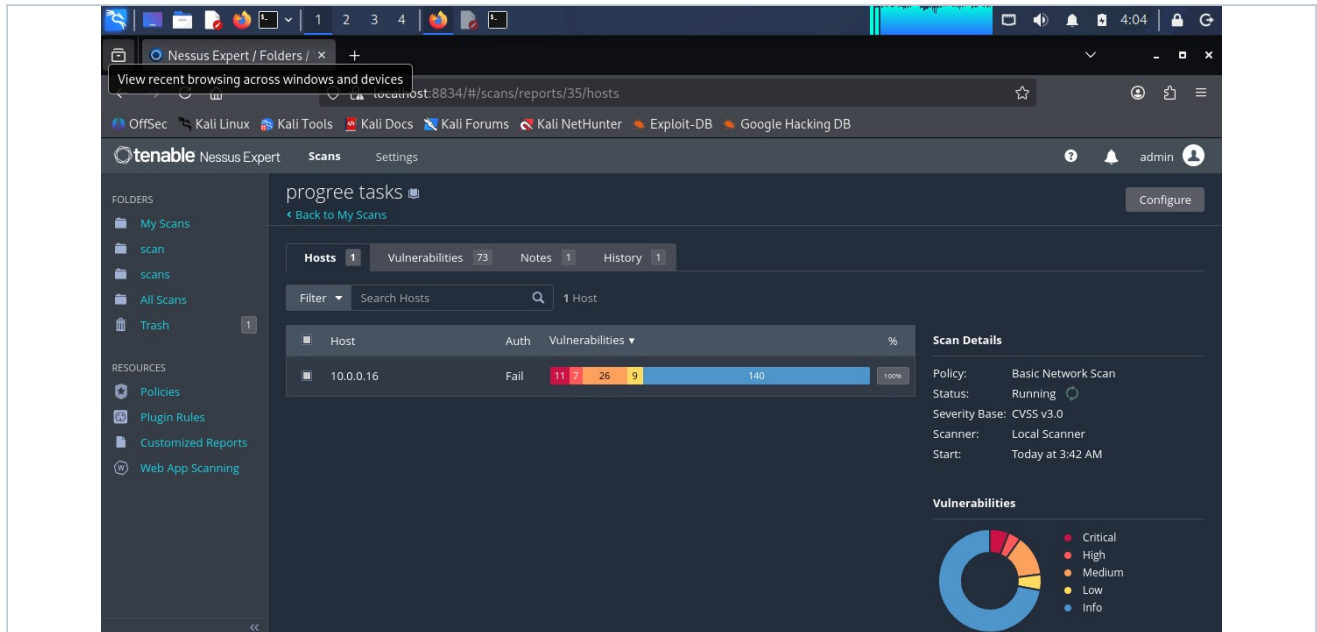


Figure 7. Nessus scan "progree tasks" (Basic Network Scan) in progress; counts are provisional.

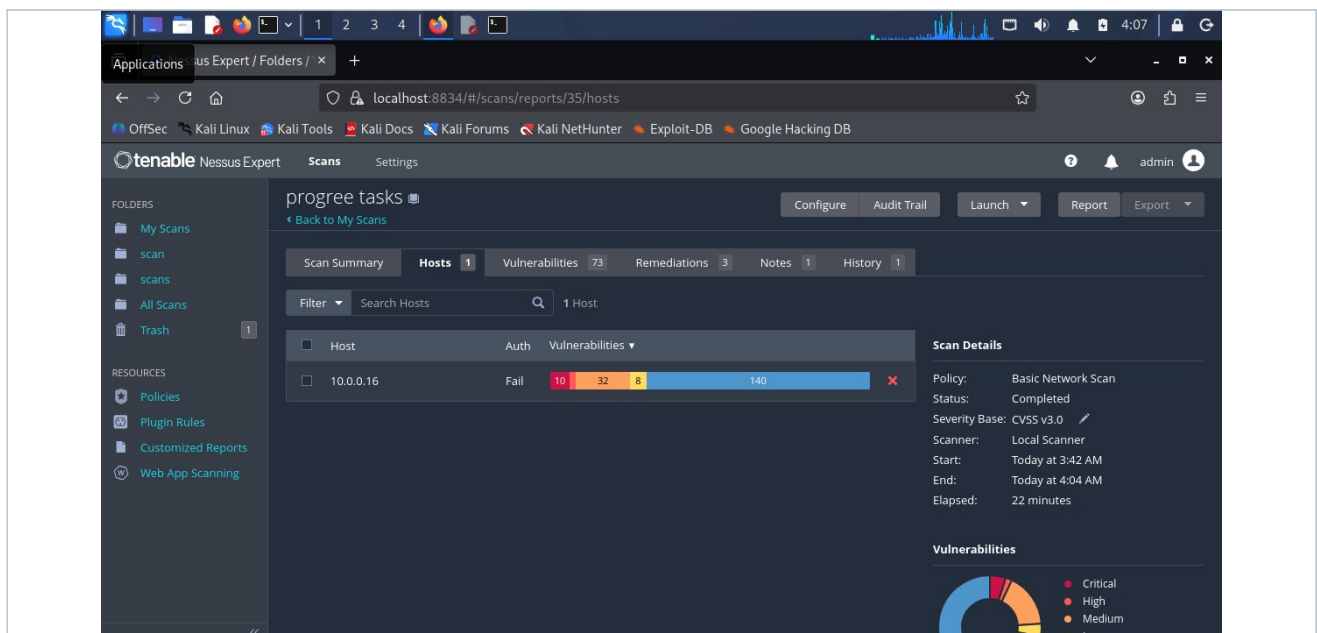


Figure 8. Completed Nessus scan: 10 Critical, 3 High, 32 Medium, 8 Low and 140 Info findings in 22 minutes.

Table 2. Nessus findings by severity

Severity	Count	Share	Examples
Critical	10	5.2%	Bind shell, UnrealIRCd backdoor, VNC weak password, Debian weak keys, SSLv2/v3, end-of-life OS and Tomcat
High	3	1.6%	Tomcat AJP "Ghostcat", rlogin, rsh
Medium	32	16.6%	Readable NFS shares, Samba Badlock, BIND flaws, SMB signing not required, Telnet, weak TLS and certificates
Low	8	4.1%	Weak SSH algorithms, ICMP timestamp, Logjam, anonymous cipher suites, X server detection
Info	140	72.5%	Service detection, open ports, OS fingerprinting, host details
Total	193	100%	Some findings repeat across ports (for example SMTP on 25 and PostgreSQL on 5432)

The Critical and High findings

Plugin	Finding (port)	CVSS v3	What it means
51988	Bind Shell Backdoor Detection (1524/tcp)	9.8	Anyone can connect and get a root shell. Nessus confirmed uid=0(root).
46882	UnrealIRCd Backdoor Detection (6667/tcp), CVE-2010-2075	10.0	The IRC server package was tampered with, so remote commands can be run.
61708	VNC Server 'password' Password (5900/tcp)	10.0	Nessus logged in with the password "password" and gained full desktop control.
32314, 32321	Debian OpenSSH/OpenSSL weak random number generator (22, 25, 5432), CVE-2008-0166	10.0	The keys come from a small, predictable set and can be brute-forced.
171340	Apache Tomcat SEoL <= 5.5.x (8180/tcp)	10.0	End of life since 30 Sep 2012, so there are no patches.
201352	Canonical Ubuntu Linux SEoL 8.04.x	10.0	The operating system is no longer supported.
20007	SSL Version 2 and 3 Protocol Detection (25, 5432)	n/a	Broken protocols are switched on (see POODLE and DROWN below).
134862	Apache Tomcat AJP Request Injection "Ghostcat" (8009/tcp), CVE-2020-1938, CVE-2020-1745	9.8	Can allow reading application files and possibly running code.
10205, 10245	rlogin and rsh service detection (513, 514), CVE-1999-0651	7.5	Clear-text, trust-based remote login that can be abused or sniffed.

3.6 Checking versions against known CVEs

I matched the detected versions to the CVE references that Nessus and Nikto gave me. Two extra entries, vsftpd 2.3.4 and distcc, come from public CVE lists. Nessus did not report them in this scan, so I marked them "verify" and did not try to exploit them.

Table 3. Software versions and their known vulnerabilities

Service (port)	Detected version	CVE / reference	Source	Risk
UnrealIRCd (6667)	Backdoored build	CVE-2010-2075	Nessus 46882	Critical
OpenSSH / OpenSSL (22, 25, 5432)	OpenSSH 4.7p1 Debian 8ubuntu1	CVE-2008-0166	Nessus 32314, 32321	Critical
Tomcat (8180)	5.5 (EOL 2012)	End of life	Nessus 171340	Critical
Ubuntu (all)	8.04 Hardy	End of life	Nessus 201352	Critical
Tomcat AJP (8009)	AJP13 connector	CVE-2020-1938, CVE-2020-1745	Nessus 134862	High
rlogin / rsh (513, 514)	netkit-rsh	CVE-1999-0651	Nessus 10205, 10245	High
vsftpd (21)	2.3.4	CVE-2011-2523 (backdoored release); verify	Public CVE list	High
distccd (3632)	distcc v1, GNU 4.2.4	CVE-2004-2687 (command execution); verify	Public CVE list	High
Samba (139, 445)	3.X to 4.X	CVE-2016-2118 (Badlock)	Nessus 90509	Medium

Service (port)	Detected version	CVE / reference	Source	Risk
ISC BIND (53)	9.4.2	CVE-2020-8616, CVE-2020-8617, CVE-2020-8622, CVE-2008-1447	Nessus 136769, 136808, 139915, 33447	Medium
Postfix SMTP (25)	Postfix smtpd	CVE-2011-0411 and five related STARTTLS CVEs	Nessus 52611	Medium
Apache (80)	2.2.8 (current is 2.4.x)	Outdated; TRACE CVE-2003-1567, CVE-2004-2320, CVE-2010-0386	Nikto 600050; Nessus 11213	Medium
PHP (80)	5.2.4-2ubuntu5.10	Outdated (current is 8.x)	Nikto 600625	Medium
TLS on 25 and 5432	SSLv2/v3, TLS 1.0, weak ciphers	CVE-2014-3566 (POODLE), CVE-2016-0800 (DROWN), CVE-2015-0204 (FREAK), CVE-2016-2183 (SWEET32), CVE-2013-2566 / CVE-2015-2808 (RC4), CVE-2015-4000 (Logjam)	Nessus	Medium
NFS (2049)	NFS v2 to v4	World-readable export (misconfiguration)	Nessus 42256	Medium

3.7 The web server (Nikto)

Nikto 2.6.1 scanned the web server on port 80 in 31 seconds. It sent 8,175 requests, had no errors and reported 31 findings (Figure 9). I grouped them by theme in Table 4.

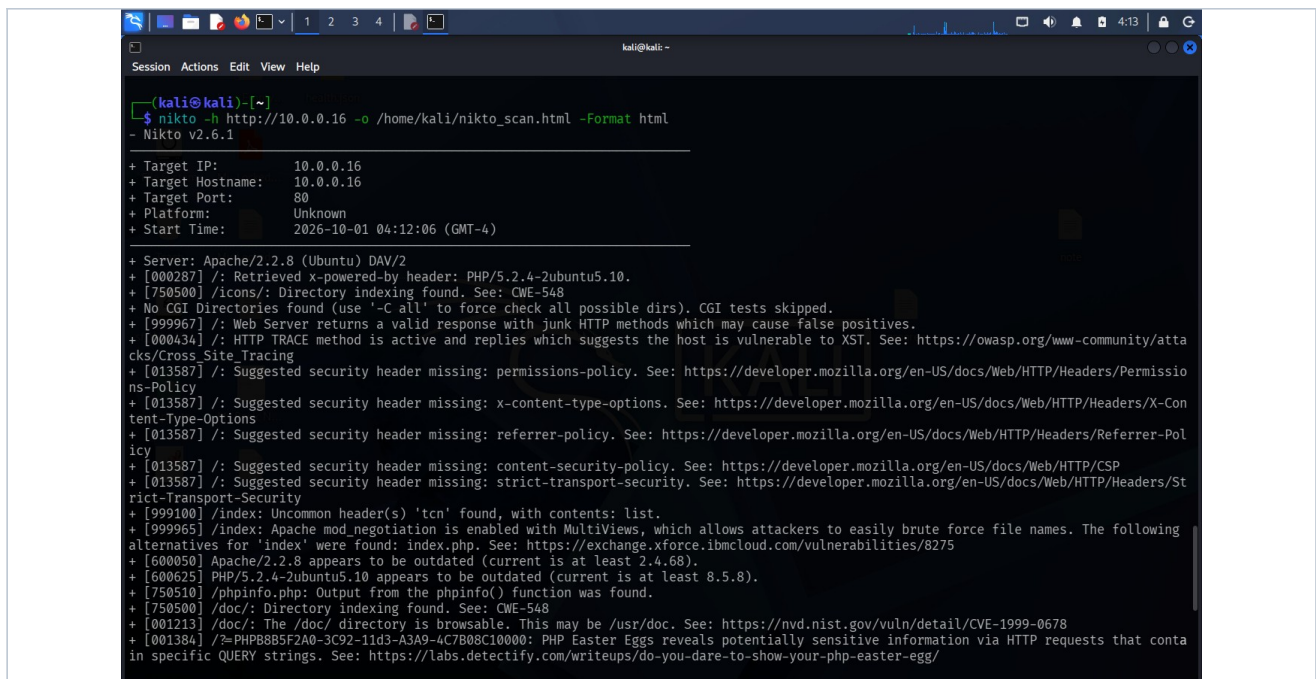


Figure 9. Nikto 2.6.1 web server scan of http://10.0.0.16 with HTML report output.

Table 4. Nikto findings grouped by theme

Theme	Finding	Risk
Information exposure	/phpinfo.php shows PHP and server configuration; PHP "Easter egg" query strings reveal information; the X-Powered-By header gives away PHP/5.2.4-2ubuntu5.10	Medium
Exposed administration	phpMyAdmin files are reachable (ChangeLog, README, Documentation.html, changelog.php). The ETag leaks the inode of /phpMyAdmin/ChangeLog (CVE-2003-1418)	Medium

Theme	Finding	Risk
Directory indexing	/icons/, /doc/ and /test/ can be browsed (CWE-548); /doc/ is a browsable documentation folder (CVE-1999-0678)	Low
Risky HTTP method	HTTP TRACE is enabled, which points to exposure to cross-site tracing (XST)	Medium
Missing security headers	Permissions-Policy, X-Content-Type-Options, Referrer-Policy, Content-Security-Policy and Strict-Transport-Security are not set	Low
Outdated software	Apache/2.2.8 and PHP/5.2.4 are flagged as outdated	Medium
Configuration	mod_negotiation with MultiViews is on (makes file names easier to guess); an uncommon "tcn" header; the default /icons/README is present	Low

Nikto does not give severity ratings, so the ratings in Table 4 are my own judgement of the likely impact. One more note: the URL `"/phpinfo.php?VARIABLE=<script>..."` in the Nikto report is just a probe used to find `phpinfo()`. On its own it is not a confirmed cross-site scripting flaw.

3.8 How I ranked the problems

To decide what to fix first, I asked three questions about each finding: how high is the CVSS score, how easy is it to abuse (no password, no user action needed), and how exposed is the service. Table 5 shows the order I ended up with.

Table 5. Triage and fix priority

Priority	Findings	Why	What I would do	Timeframe
Critical	Bind shell (1524); UnrealIRCd backdoor (6667); VNC password (5900); Debian weak keys (22, 25, 5432)	Remote, no login needed, full takeover	Isolate the host. Remove the bind shell and reinstall UnrealIRCd from verified checksums. Set a strong VNC password or close the port. Regenerate all keys and certificates.	Right away (0 to 24 hours)
High	Tomcat Ghostcat (8009); rlogin/rsh (513, 514); vsftpd 2.3.4 and distcc (verify); end-of-life OS and Tomcat	Remote code or file access; no vendor patches	Disable AJP or bind it to localhost and upgrade Tomcat. Disable rlogin, rsh and rexec. Replace or verify vsftpd and distccd. Plan a rebuild on a supported OS.	Within 7 days
Medium	NFS world-readable; Samba Badlock and SMB signing; BIND flaws; Telnet; phpinfo and phpMyAdmin exposure; TRACE	Information leaks and a way to move sideways	Limit NFS exports to named hosts. Upgrade Samba and BIND and require SMB signing. Replace Telnet with SSH. Remove phpinfo.php and restrict phpMyAdmin to admin hosts. Turn TRACE off.	Within 30 days
Low	Weak TLS ciphers and certificates, SSH CBC and weak MACs, missing headers, directory indexing, version banners	Hardening gaps with limited direct impact	Disable SSLv2/v3 and TLS 1.0, drop weak ciphers, replace self-signed or expired certificates, add security headers, switch off indexing and hide version banners.	Within 90 days

3.9 What I would recommend

- **Contain first.** Take the host off any shared network, because several services give unauthenticated root access.
- **Remove what is not needed.** Close or uninstall rlogin, rsh, rexec, Telnet, distccd, the bind shell, Java RMI, DRb and the test web content. Every closed port is one less way in.

- **Replace clear-text protocols** with SSH/SFTP and HTTPS/FTPS, and turn off SSLv2, SSLv3 and TLS 1.0.
- **Rebuild instead of patching.** A machine running Ubuntu 8.04, PHP 5.2.4 and Apache 2.2.8 cannot realistically be secured by patches. A fresh install on a supported release, with only the needed services, is the right fix.
- **Scan again with credentials.** A credentialed Nessus scan gives more accurate patch results and fewer false positives. After fixing, I would repeat all three scans to confirm each problem is closed.

4. Task 3: Low-Level Network Packet Inspection & Protocol Analysis

4.1 What the task asked for

This task was about watching network traffic closely: following the TCP handshake, looking at individual streams, spotting clear-text logins and picking out a data exfiltration. I opened Wireshark 4.6.6 on Kali (Figure 10) and started it with sudo so it could capture on eth0 (Figure 11). Everything below comes from a single capture, wireshark_eth0JWT5V3.pcapng, between 10.0.0.4 (my Kali machine) and 10.0.0.16 (the target).

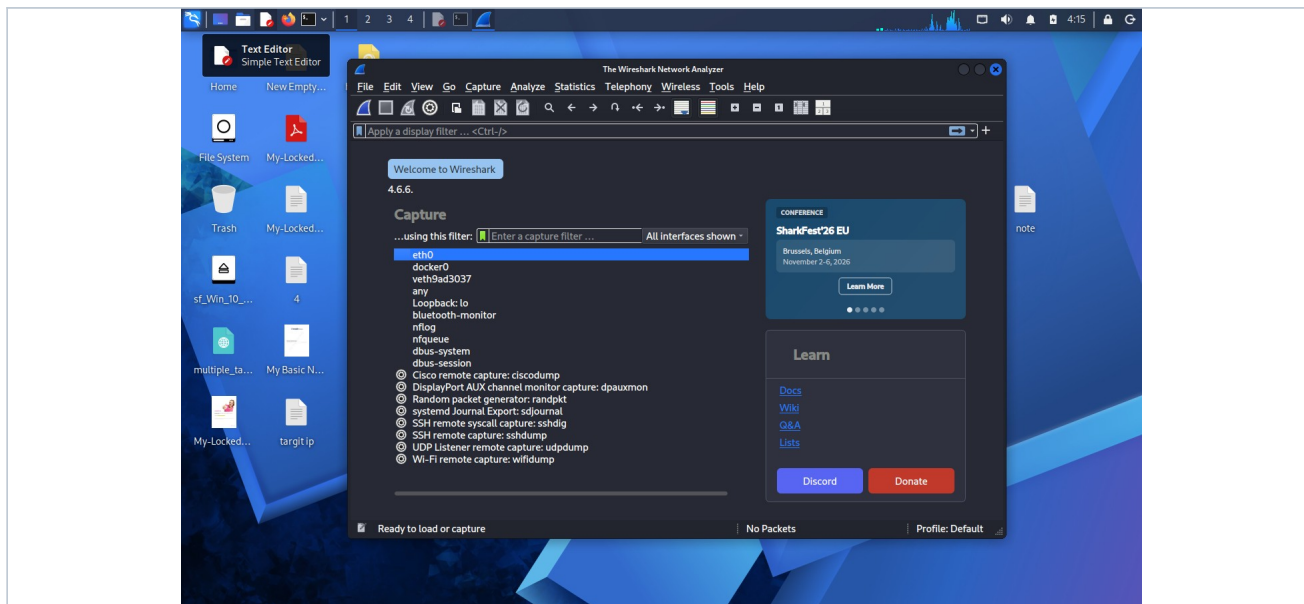


Figure 10. Wireshark 4.6.6 welcome screen on Kali Linux with eth0 available for capture.

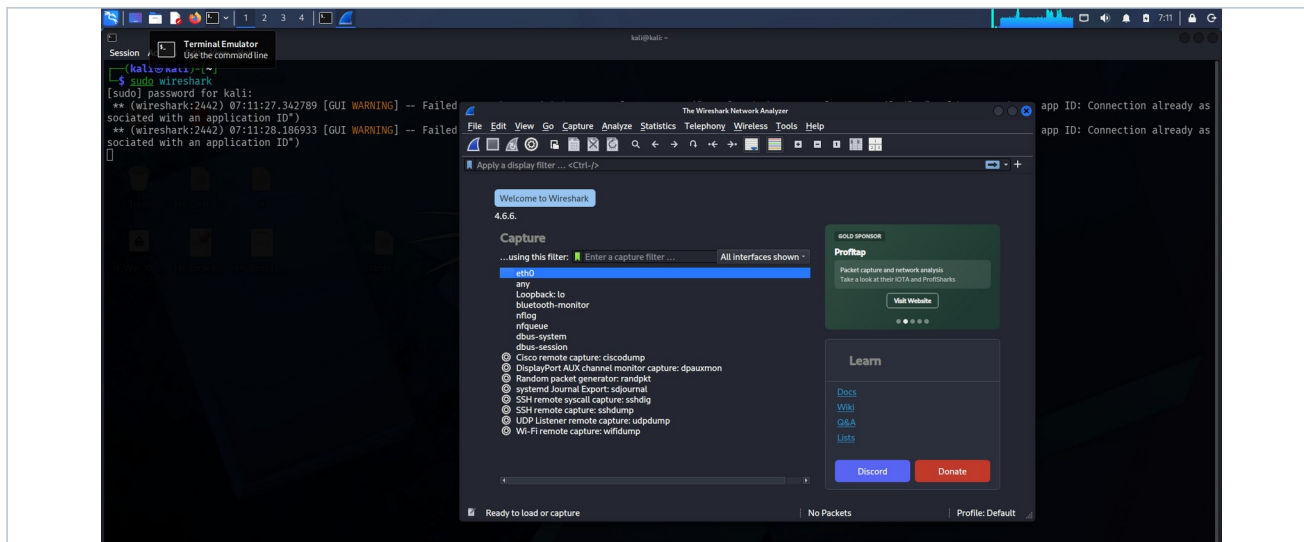


Figure 11. Wireshark launched with sudo from the terminal; eth0 selected as the capture interface.

4.2 What a port scan looks like on the wire

While Nmap was probing the target, Wireshark filled up with short TCP packets from 10.0.0.16 back to 10.0.0.4, all marked [RST, ACK] with no data, each from a different source port (749, 2710, 10002, 16001 and so on) (Figure 12). A closed port answers an unwanted connection attempt with a reset, so a flood of these is the classic fingerprint of a **TCP port scan**. The capture already held 87,702 packets at that point. A monitoring system should raise an alert when one host sends this many resets in such a short time.

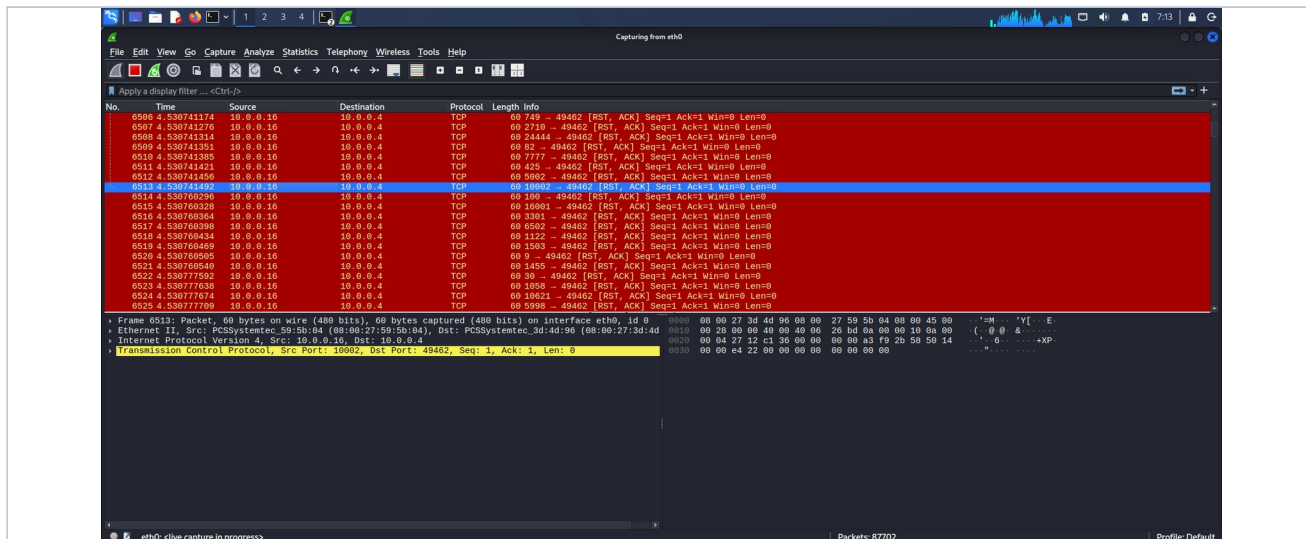


Figure 12. Port-scan traffic in Wireshark: a burst of TCP RST/ACK replies from 10.0.0.16 to closed ports.

4.3 Everyday traffic: ICMP, DNS and ARP

The capture also shows ordinary network chatter. ICMP echo requests and replies pass between the lab hosts and an outside address (142.250.190.236), with TTL 64, which is typical of Linux (Figures 13 and 14). The target sends a DNS query to 192.168.0.1 on UDP port 53 (Figure 13). ARP messages such as "Who has 10.0.0.16? Tell 10.0.0.4" match IP addresses to MAC addresses, and the reply confirms that 10.0.0.16 is at 08:00:27:59:5b:04 (Figure 14). ARP and DNS carry no authentication, so on a hostile network they can be faked. I include them as a baseline to compare with the unusual traffic later.

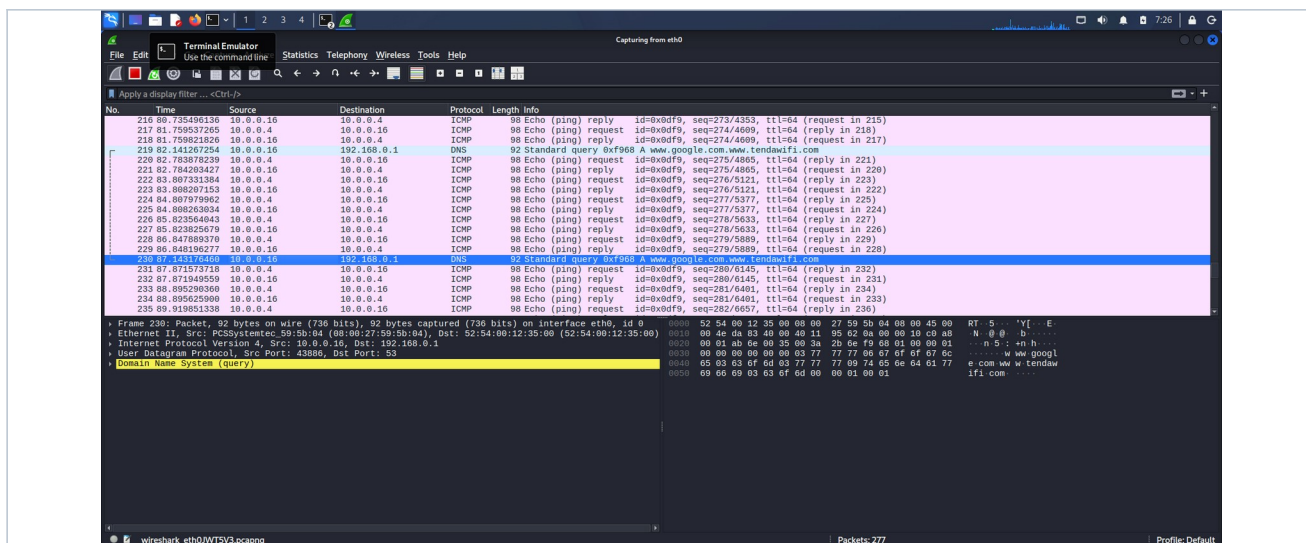


Figure 13. ICMP echo request/reply traffic and a DNS query from 10.0.0.16 to 192.168.0.1.

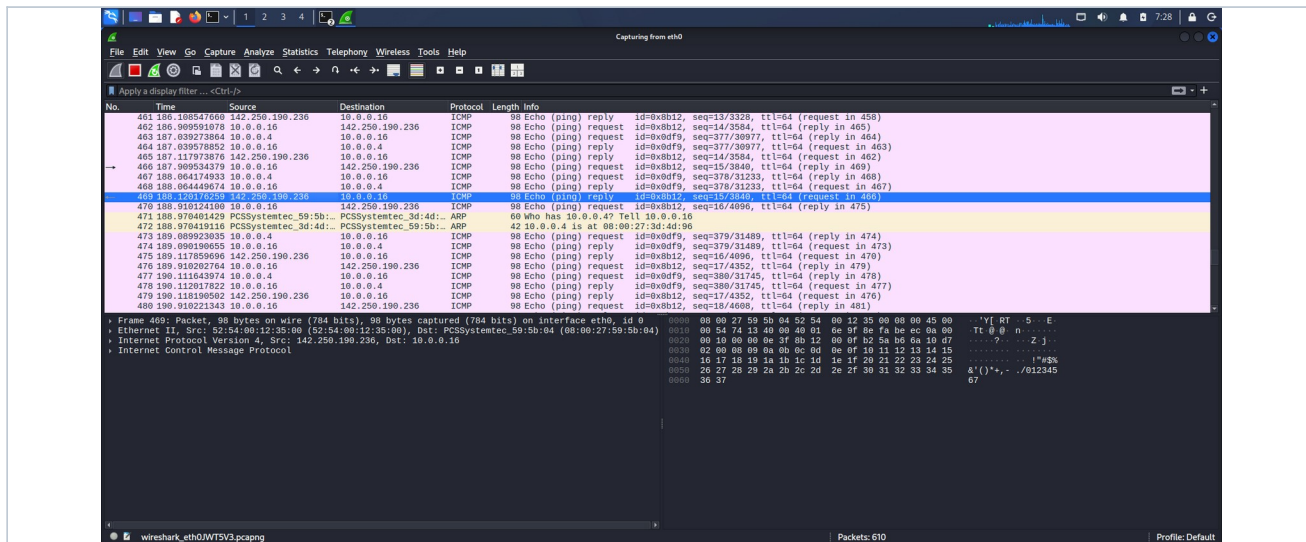


Figure 14. ICMP echo traffic together with ARP address resolution between 10.0.0.4 and 10.0.0.16.

4.4 The TCP three-way handshake

Figure 15 shows one complete connection from my machine (port 45724) to the web server on port 80:

- **SYN** (packet 1109): my machine asks to connect. Seq=0, window 64240, MSS 1460, SACK permitted.
- **SYN/ACK** (packet 1110): the server agrees. Ack=1, window 5792.
- **ACK** (packet 1111): I confirm, and the connection is open.
- **FIN/ACK, FIN/ACK, ACK** (packets 1112 to 1114): both sides close the connection politely.

This is what a healthy session looks like. The port scan in Figure 12 is different, because it never gets as far as a finished handshake: the target simply answers with a reset.

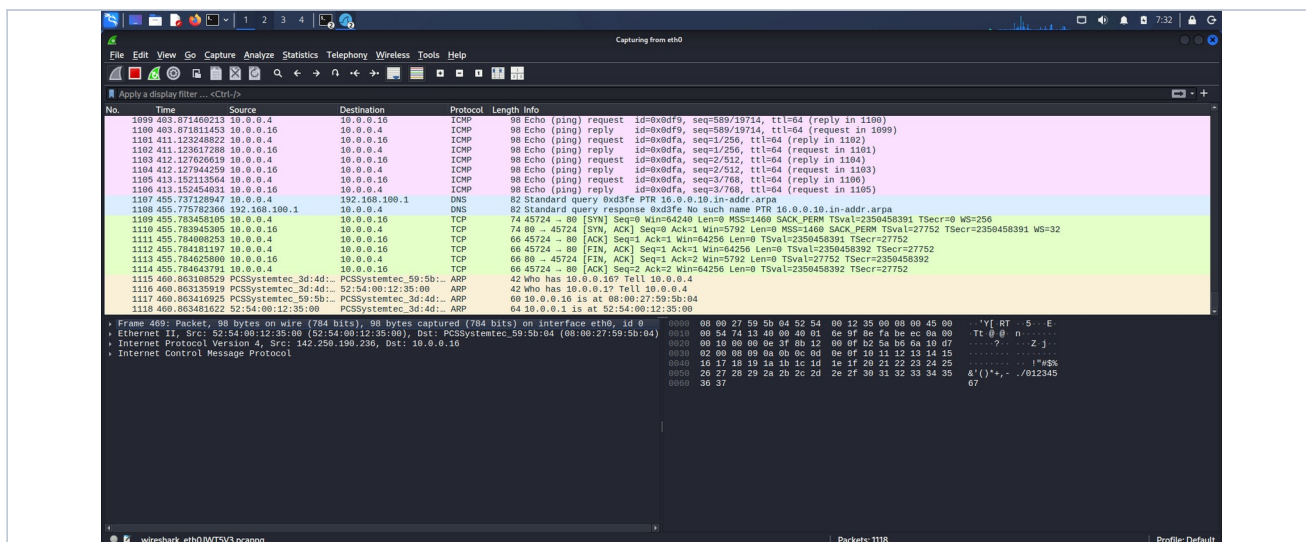


Figure 15. TCP three-way handshake (SYN, SYN/ACK, ACK) to port 80 followed by a graceful FIN/ACK close.

4.5 Plain HTTP

About 493 seconds into the capture there is another handshake, followed by an **HTTP GET / HTTP/1.1** request and an **HTTP/1.1 200 OK (text/html)** reply of 1,147 bytes (Figure 16). HTTP is not encrypted, so the page, headers, cookies and any form data would be visible to anything on the path. This ties back to Nikto, which reported that Strict-Transport-Security is missing (Section 3.7).

4.6 Passwords sent in the clear over FTP

- My first attempt, **USER kali** with **PASS msfadmin**, was refused with **530 Login incorrect**.
- My second attempt, **USER msfadmin** (packet 1160, Figure 18) with **PASS msfadmin** (packet 1164, Figure 19), worked: **230 Login successful**.
- The session then ran SYST (215 UNIX Type: L8) and FEAT, and ended with QUIT and 221 Goodbye.

The image shows a Wireshark packet capture of an FTP session. The packet list on the left shows packets 1189 to 1193. The packet details pane on the right shows the selected packet (1189) as an FTP Response (220) from 10.0.0.16 to 10.0.0.4. The packet bytes pane on the right shows the raw data of the response.

No.	Time	Source	Destination	Protocol	Length	Info
1189	537.043695938	10.0.0.16	10.0.0.4	FTP	86	Response: 220 (vsFTPd 2.3.4)
1149	550.963171748	10.0.0.4	10.0.0.16	FTP	77	Request: USER kali
1142	550.963596319	10.0.0.16	10.0.0.4	FTP	100	Response: 331 Please specify the password.
1144	556.935187489	10.0.0.4	10.0.0.16	FTP	81	Request: PASS msfadmin
1146	560.184291368	10.0.0.16	10.0.0.4	FTP	80	Response: 530 Login incorrect.
1149	576.759979257	10.0.0.16	10.0.0.4	FTP	76	Response: 500 OOPS:
1150	576.759979489	10.0.0.16	10.0.0.4	FTP	96	Response: vsf_sysutil_recv_peek: no data
1151	576.759979541	10.0.0.16	10.0.0.4	FTP	68	Response:
1158	589.929529633	10.0.0.16	10.0.0.4	FTP	86	Response: 220 (vsFTPd 2.3.4)
1159	589.929531317	10.0.0.4	10.0.0.16	FTP	81	Request: USER msfadmin
1162	589.975864966	10.0.0.16	10.0.0.4	FTP	100	Response: 331 Please specify the password.
1164	612.468563051	10.0.0.4	10.0.0.16	FTP	81	Request: PASS msfadmin
1165	612.470237017	10.0.0.16	10.0.0.4	FTP	89	Response: 230 Login successful.
1167	612.470356977	10.0.0.4	10.0.0.16	FTP	72	Request: SYST
1168	612.470563967	10.0.0.16	10.0.0.4	FTP	85	Response: 215 UNIX Type: L8
1169	612.470682264	10.0.0.16	10.0.0.16	FTP	72	Request: FEAT
1170	612.470847807	10.0.0.16	10.0.0.4	FTP	81	Response: 211-Features:
1171	612.470847179	10.0.0.16	10.0.0.4	FTP	73	Response: EPRT
1172	612.470847211	10.0.0.16	10.0.0.4	FTP	73	Response: EPSV
1174	612.471194715	10.0.0.16	10.0.0.4	FTP	124	Response: MDTM
1182	669.494345099	10.0.0.4	10.0.0.16	FTP	72	Request: QUIT
1183	669.495379039	10.0.0.16	10.0.0.4	FTP	80	Response: 221 Goodbye.

File Transfer Protocol (FTP): Protocol

Packets: 1189 - Displayed: 22 (1.9%) - Selected: 3 (0.3%)

Profile: Default

Muhammad Ali Mobeen • B7/9 | Isolated lab assessment • For educational use | 18

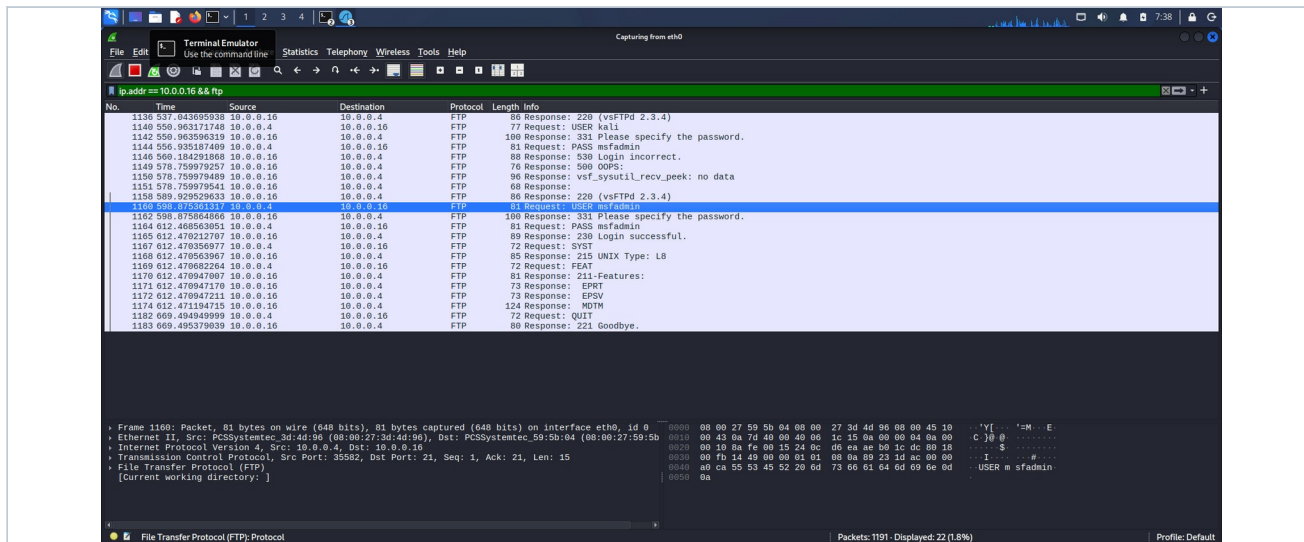


Figure 18. Packet 1160: FTP "USER msfadmin" visible in plain text in the packet bytes.

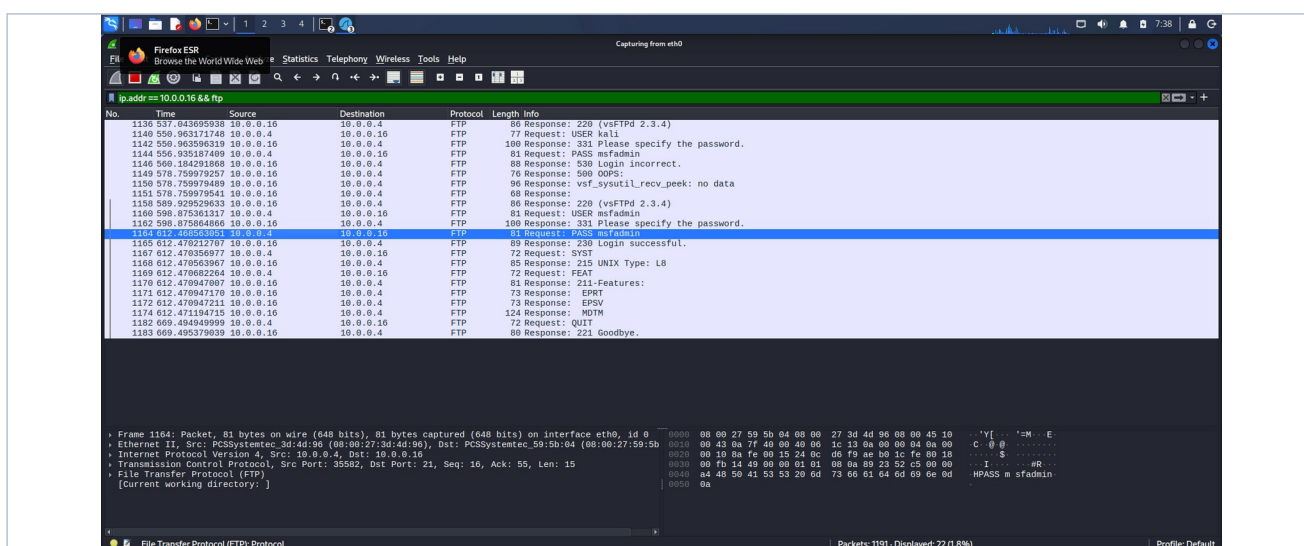


Figure 19. Packet 1164: FTP "PASS msfadmin" visible in plain text, followed by 230 Login successful.

4.7 A simulated data exfiltration

To create a realistic exfiltration pattern, I generated a **10 MB test file** from /dev/zero with dd (10,485,760 bytes). My first attempt to send it to 10.0.0.16 with netcat on port 4444 failed with "Connection refused", which tells me nothing was listening on that port (Figure 20). I then uploaded the file through the open FTP service with the commands binary and put. The transfer finished with **226 Transfer complete** at about 78.66 MiB/s (Figure 21).

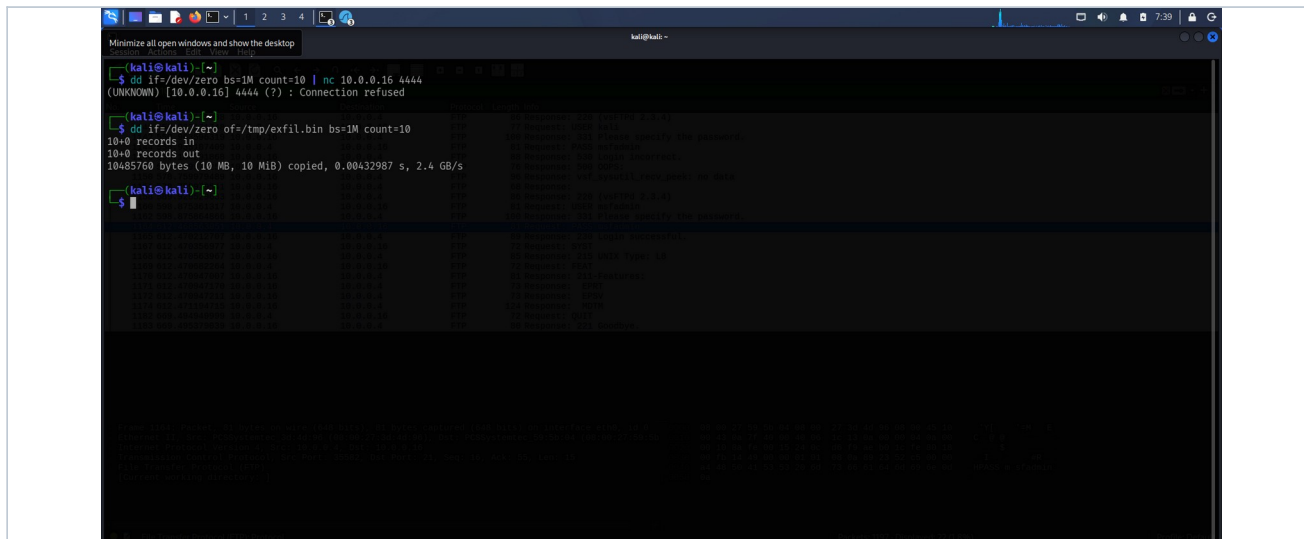


Figure 20. Exfiltration simulation, step 1: 10 MB test file created with dd; netcat to port 4444 refused.

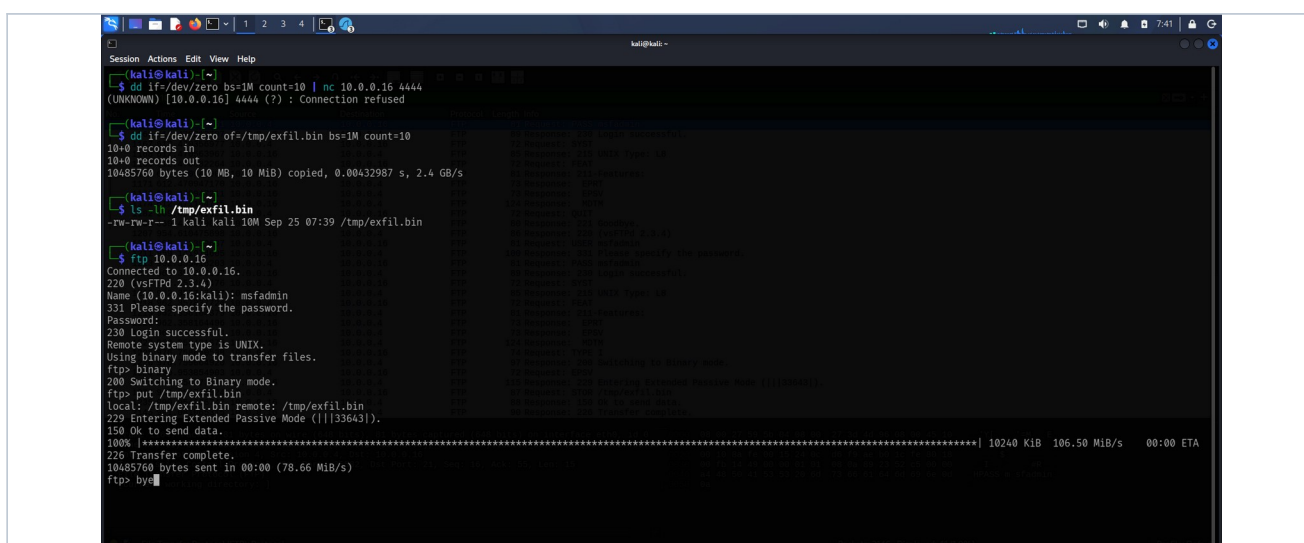


Figure 21. Exfiltration simulation, step 2: 10 MB file uploaded over FTP; "226 Transfer complete".

In Wireshark I filtered for large segments (**ip.src == 10.0.0.4 && ip.dst == 10.0.0.16 && tcp.len > 500**), which showed 266 of 2,152 packets (12.4%). These are FTP-DATA packets of up to about 65,160 bytes, all labelled "(EPASV) (STOR /tmp/exfil.bin)", sent in one direction in under a second, around 991 seconds into the capture. Several are marked **[TCP Window Full]**, which means the receiver's buffer filled up under the burst (Figure 22).

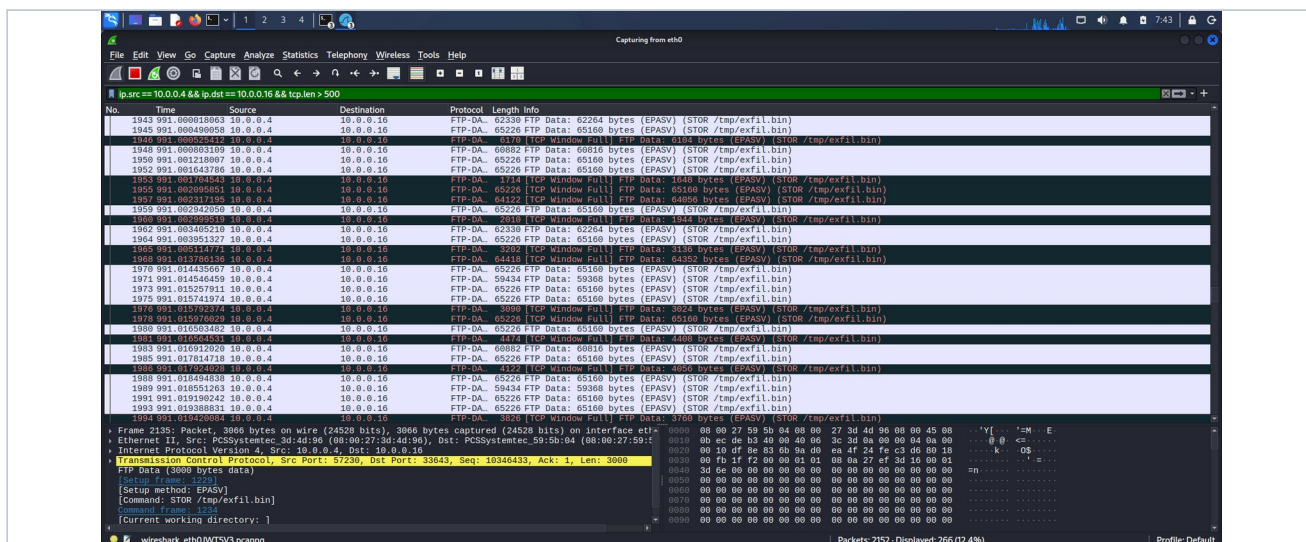


Figure 22. Wireshark filter for large segments from 10.0.0.4 to 10.0.0.16 showing FTP-DATA for STOR /tmp/exfil.bin.

Following the TCP stream (stream 7) shows the whole 10 MB conversation as one-way traffic, with no data coming back from the server (Figure 23). The content appears as dots because the file is made of null bytes from /dev/zero. In a real incident the same view would show the stolen content directly, since FTP does not encrypt its data channel.

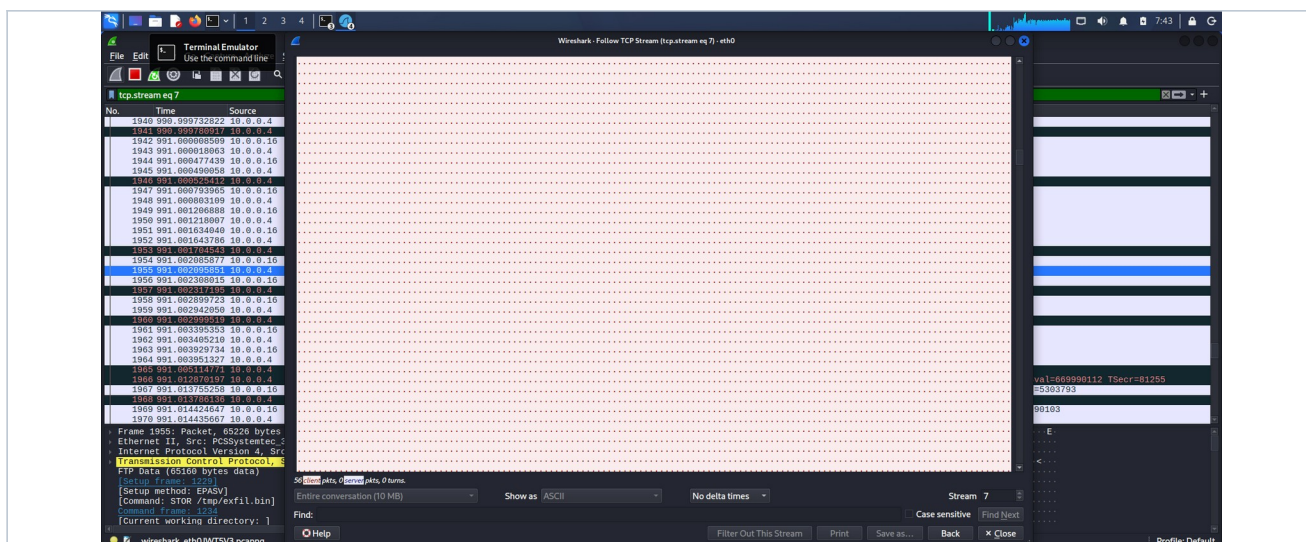


Figure 23. Follow TCP Stream (stream 7): the entire 10 MB conversation flowing in one direction only.

Signs that pointed to exfiltration

- One large, one-way transfer of about 10 MB in a very short time.
- Data sent to an unusual FTP data port (33643, EPASV) right after a login with default credentials.
- Maximum-size segments sent back to back with TCP Window Full flags, which looks like bulk copying and not interactive use.
- An earlier refused connection to port 4444, a port often used for backdoor listeners.

4.8 Findings and what I would do about them

Table 6. Packet analysis findings

Finding	Evidence	Risk	Recommendation
FTP credentials sent in clear text	Figures 17 to 19	High	Replace FTP with SFTP or FTPS and block port 21 where possible.

Finding	Evidence	Risk	Recommendation
Default login accepted (msfadmin / msfadmin)	Figures 18, 19	High	Remove default accounts; require strong passwords and multi-factor authentication.
Large unencrypted FTP upload (exfiltration pattern)	Figures 21 to 23	High	Use egress filtering and DLP; alert on large outbound transfers; deploy IDS/NDR.
HTTP traffic not encrypted	Figure 16	Medium	Enforce HTTPS with HSTS and modern TLS.
Port-scan signature (RST/ACK burst)	Figure 12	Medium	Add IDS rules that detect scans and rate-limit resets.
Failed login followed by a successful one	Figure 17	Medium	Enable account lockout and alert on repeated authentication failures.
Server version banner shown (vsFTPD 2.3.4)	Figure 17	Low	Hide version banners and upgrade to a supported FTP server.
ARP and DNS are unauthenticated	Figures 13, 14	Low	Use dynamic ARP inspection, DNSSEC and network segmentation.

5. Conclusion and What I Learned

The two tasks taught me the same lesson from two directions. In Task 2 I saw, from the outside, that 10.0.0.16 runs old, backdoored and badly configured services that would let an attacker take full control. In Task 3 I saw what someone watching the wire would learn: FTP and HTTP give away passwords and data in clear text, and a bulk transfer stands out clearly once the right filters are applied.

5.1 Skills I practised

- Installing and using Nessus, Nmap, Nikto and Wireshark in an isolated lab.
- Mapping open ports and matching service versions to CVEs and end-of-life information.
- Ranking findings by severity, ease of exploitation and exposure, and turning them into an action plan.
- Reading TCP handshakes, using display filters, following streams and recognising scan and exfiltration patterns.
- Documenting my work clearly, with a screenshot as evidence for each step.

5.2 Limits of my work and next steps

- My Nessus scan was **unauthenticated**. A credentialed scan would give deeper, more accurate patch results.
- Nmap found 30 open ports but my screenshot shows only the first 22. I should save the full Nmap output to a file next time.
- I identified vulnerabilities but did not exploit them, on purpose, to keep the assessment safe. The items marked "verify" in Table 3 still need to be confirmed by hand.
- In a real environment I would repeat every scan and capture after the fixes, to prove the problems are closed.

Thank you. I am grateful to the Progree team for the guidance and the practical tasks during this internship. They gave me a safe way to learn how real security assessments are done.

Appendix A: Evidence Index

I renamed every screenshot by task and step. This table links each figure in the report to its file name.

Fig.	Task	Description	File name
1	Task 2	Installing the zypper prerequisite on Kali Linux before installing Nessus.	T2_01_nessus_prerequisites_zypper_install.png
2	Task 2	Installing Nessus 10.12.4 with dpkg; all cryptographic self-tests report Pass.	T2_02_nessus_install_dpkg.png
3	Task 2	Disposable e-mail address used to receive the Nessus trial activation code.	T2_03_tempmail_for_nessus_activation.png
4	Task 2	Nessus Expert first-run screen; plugins are compiling (functionality limited until complete).	T2_04_nessus_welcome_plugins_compiling.png
5	Task 2	Nessus host discovery of 10.0.0.0/24 listing live hosts, including the target 10.0.0.16.	T2_05_nessus_host_discovery_10.0.0.0-24.png
6	Task 2	Nmap service-version scan (nmap -sV -p- 10.0.0.16) listing open ports and software versions.	T2_06_nmap_open_ports_service_versions.png
7	Task 2	Nessus scan "progree tasks" (Basic Network Scan) in progress; counts are provisional.	T2_07_nessus_scan_running.png
8	Task 2	Completed Nessus scan: 10 Critical, 3 High, 32 Medium, 8 Low and 140 Info findings in 22 minutes.	T2_08_nessus_scan_completed_summary.png
9	Task 2	Nikto 2.6.1 web server scan of http://10.0.0.16 with HTML report output.	T2_09_nikto_web_scan_terminal.png
10	Task 3	Wireshark 4.6.6 welcome screen on Kali Linux with eth0 available for capture.	T3_01_wireshark_welcome_screen.png
11	Task 3	Wireshark launched with sudo from the terminal; eth0 selected as the capture interface.	T3_02_wireshark_launch_sudo_eth0.png
12	Task 3	Port-scan traffic in Wireshark: a burst of TCP RST/ACK replies from 10.0.0.16 to closed ports.	T3_03_nmap_scan_seen_in_wireshark_RST_packets.png
13	Task 3	ICMP echo request/reply traffic and a DNS query from 10.0.0.16 to 192.168.0.1.	T3_04_icmp_ping_and_dns_query.png
14	Task 3	ICMP echo traffic together with ARP address resolution between 10.0.0.4 and 10.0.0.16.	T3_05_icmp_and_arp_resolution.png
15	Task 3	TCP three-way handshake (SYN, SYN/ACK, ACK) to port 80 followed by a graceful FIN/ACK close.	T3_06_tcp_three_way_handshake_syn_synack_ack.png
16	Task 3	Unencrypted HTTP: GET / request and 200 OK (text/html) response in clear text.	T3_07_http_get_cleartext_200ok.png
17	Task 3	FTP session filtered with ip.addr == 10.0.0.16 && ftp, showing the failed and successful logins.	T3_08_ftp_filter_session_overview.png
18	Task 3	Packet 1160: FTP "USER msfadmin" visible in plain text in the packet bytes.	T3_09_ftp_cleartext_username.png
19	Task 3	Packet 1164: FTP "PASS msfadmin" visible in plain text, followed by 230 Login successful.	T3_10_ftp_cleartext_password.png
20	Task 3	Exfiltration simulation, step 1: 10 MB test file created with dd; netcat to port 4444 refused.	T3_11_exfil_simulation_dd_nc_refused.png
21	Task 3	Exfiltration simulation, step 2: 10 MB file uploaded over FTP; "226 Transfer complete".	T3_12_ftp_upload_exfil_10MB_complete.png

Fig.	Task	Description	File name
22	Task 3	Wireshark filter for large segments from 10.0.0.4 to 10.0.0.16 showing FTP-DATA for STOR /tmp/exfil.bin.	T3_13_wireshark_exfil_stor_packets_filter.png
23	Task 3	Follow TCP Stream (stream 7): the entire 10 MB conversation flowing in one direction only.	T3_14_follow_tcp_stream_10MB_exfil.png

Appendix B: Task Coverage

Task	Title	Coverage in this report
Task 1	Professional LinkedIn Announcement	Not part of this report (a separate public post).
Task 2	Host & Network Vulnerability Assessment Lab	Covered in Section 3 (Figures 1 to 9).
Task 3	Low-Level Network Packet Inspection & Protocol Analysis	Covered in Section 4 (Figures 10 to 23).
Task 4	Mini Project: Threat Modeling & Incident Response Blueprint	Not part of this report (a separate 5 to 6 page project document).

Prepared by: Muhammad Ali Mobeen | Student ID: B7/9 | Position: Cyber Security Intern, Progree